Program Security

Buffer Overflows & Memory Corruption

A Process in Memory

The Stack

- Grows **downward** (high → low addresses)
- Each function call pushes a **stack frame**
- A frame holds: local variables, saved registers, **return address**

> *The return address is the attacker's prime target.*

The Heap

- Grows **upward**
- Managed by malloc / free (libc)
- Each allocation is preceded by **heap metadata** (size, flags, pointers)

```
char *buf = malloc(64);   // allocates 64 bytes on the heap
strcpy(buf, input);       // if input > 64 bytes → overflow
free(buf);
```

> *Overflowing a heap buffer can corrupt adjacent metadata or objects.*

CPU Registers (x86-64)

Register	Role
RIP	Instruction pointer — *next* instruction
RSP	Stack pointer — top of stack
RBP	Base pointer — current frame
RAX–R15	General purpose

Exploits often aim to control **RIP**.

What is a Buffer Overflow?

A **buffer** is a contiguous block of memory holding data.

An **overflow** happens when a program writes **more bytes** than the buffer can hold, spilling into adjacent memory.

```
char buf[8];          // 8 bytes allocated
gets(buf);            // reads unlimited input — DANGEROUS
//  input: "AAAAAAAAAAAAAAAA" (16 bytes)
//  buf[8..15] overwrite whatever comes after buf
```

> *C does **not** check array bounds. The programmer is responsible.*

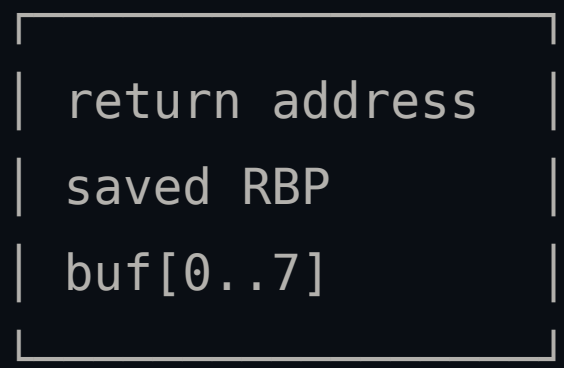
Dangerous C Functions

Function	Problem	Safe alternative
gets	no length limit	fgets
strcpy	no length check	strncpy / strlcpy
strcat	no length check	strncat / strlcat
sprintf	no length check	snprintf
scanf("%s")	no limit	scanf("%Ns", ...)

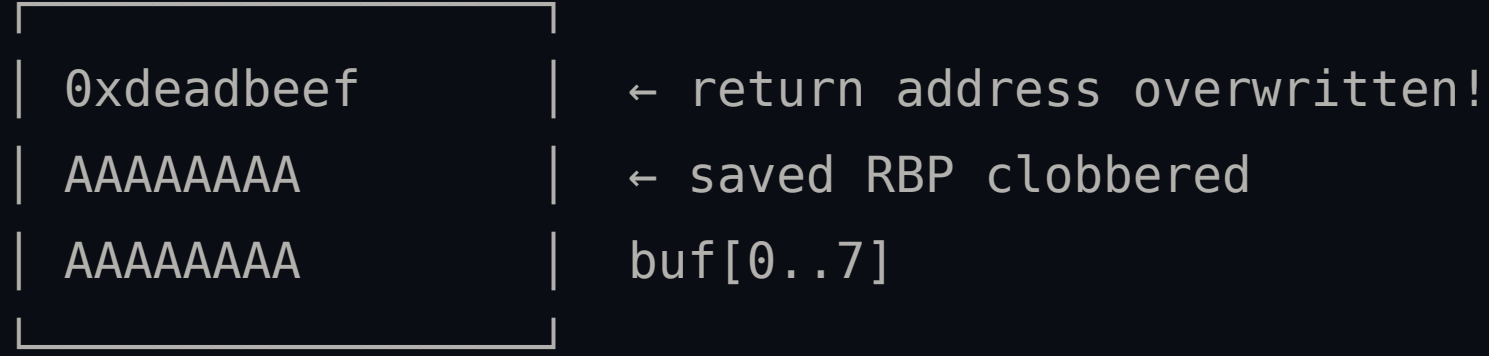
These functions are the root cause of most classic stack overflows.

Stack Buffer Overflow — Step by Step

Normal execution:



After overflow ("AAAAAAAA" + "\xef\xbe\xad\xde"):



Stack Buffer Overflow — Code Example

```c
#include <stdio.h>
#include <string.h>

void greet(char *name) {
    char buf[64];
    strcpy(buf, name);          // no length check
    printf("Hello, %s!\n", buf);
}

int main(int argc, char *argv[]) {
    greet(argv[1]);
    return 0;
}
```

```
$ ./vuln $(python3 -c "print('A'*80)")
```

Heap Buffer Overflow

Heap overflows corrupt **neighbouring allocations** or **heap metadata**.

```
struct User {
    char name[16];
    int  is_admin;
};

struct User *u = malloc(sizeof(struct User));
u->is_admin = 0;

// Overflow name → overwrite is_admin
memcpy(u->name, input, input_len);   // no bounds check
```

If input_len > 16, the is_admin field is overwritten.

What Can Go Wrong?

Crash (DoS)

The process segfaults. Service goes down.

Arbitrary Code Execution

Overwrite return address / function pointer → run shellcode or ROP chain.

Data Corruption

Flip flags, overwrite credentials, alter control flow silently.

Stack Canary

A **random value** placed between local variables and the return address.

- If the canary is modified → **abort** before returning
- Compile with: `gcc -fstack-protector-all`
- Bypass: leak the canary first, then include it in the payload

ASLR — Address Space Layout Randomisation

The OS randomises base addresses of **stack**, **heap**, **and libraries** on every run.

```
$ cat /proc/self/maps | grep libc
7f3a2b400000-... libc.so.6    # run 1
7f9c1d200000-... libc.so.6    # run 2  ← different!
```

- Makes hardcoded addresses useless
- Enabled OS-wide: /proc/sys/kernel/randomize_va_space = 2
- Bypass: **info leak** to discover the runtime base address

NX / DEP — Non-Executable Memory

Memory pages are either **writable** or **executable**, never both.

Region	Permissions
Stack	RW (no execute)
Heap	RW (no execute)
Text	RX (no write)

- Injected shellcode on the stack/heap **cannot run**
- Bypass: **Return-Oriented Programming (ROP)** — reuse existing code gadgets

The Attacker’s Checklist

To exploit a buffer overflow in a hardened binary you typically need to:

1. Find the overflow (fuzzing, source review, reverse engineering)
2. **Leak** a stack/heap/libc address (defeat ASLR)
3. **Leak or brute-force** the stack canary
4. Build a **ROP chain** or ret2libc payload (defeat NX)
5. Trigger the overflow with the crafted payload

Each defence adds a layer — none is bulletproof alone.

Why Third-Party Libraries Matter

Most programs rely on **dozens of libraries**. A vulnerability in a dependency affects **every program** that links it.

> *In 2021, Log4Shell (CVE-2021-44228) affected millions of systems through a single Java logging library.*

Common vulnerable categories:

- **Parsers** — image, audio, document, archive formats
- **Protocol implementations** — TLS, SSH, HTTP
- **Compression** — zlib, bzip2, lz4

Case Study: libsndfile — CVE-2017-8361

libsndfile reads and writes audio files (WAV, AIFF, FLAC…). Used by PulseAudio, Audacity, FFmpeg, and countless audio tools.

The vulnerability

A stack buffer overflow in aiff_read_chanmap() (src/aiff.c): the function copies channel-map entries from an AIFF CHAN chunk into a fixed-size local array **without checking the chunk size**, so a crafted AIFF file overflows the stack buffer.

- **CVE**: CVE-2017-8361 — CVSS 8.8 (High)
- **Affected**: libsndfile $\leq$ 1.0.28
- **Fixed in**: 1.0.29

libsndfile — API (the whole thing)

```c
#include <sndfile.h>

SF_INFO sfinfo = {0};

// Open — this is where the vulnerable parse happens
SNDFILE *sf = sf_open("audio.aiff", SFM_READ, &sfinfo);

// Read frames
short buf[1024];
sf_readf_short(sf, buf, 512);

// Close
sf_close(sf);
```

Three functions (sf open, sf readf *, sf close). The

libsndfile — The Vulnerable Program

```c
// player.c — links libsndfile <= 1.0.28 (vulnerable)
#include <stdio.h>
#include <sndfile.h>

int main(int argc, char *argv[]) {
    if (argc != 2) { fprintf(stderr, "Usage: %s <file>\n", ar

    SF_INFO sfinfo = {0};
    SNDFILE *sf = sf_open(argv[1], SFM_READ, &sfinfo);
    if (!sf) { fprintf(stderr, "Error: %s\n", sf_strerror(NUL

    printf("channels=%d  frames=%ld  rate=%d Hz\n",
           sfinfo.channels, (long)sfinfo.frames, sfinfo.sampl
    sf_close(sf);
    return 0;
```

The Root Cause (one slide, one function)

```
// src/aiff.c — aiff_read_chanmap() in libsndfile <= 1.0.28
static int aiff_read_chanmap(SF_PRIVATE *psf, unsigned datale
    // datalength comes straight from the file — attacker-con
    unsigned channel_map_info[SF_MAX_CHANNELS];          // f
    unsigned bytesread = 0;

    while (bytesread < datalength) {                     // ←
        psf_binheader_readf(psf, "444",
            &channel_map_info[bytesread / 12],           // ←
            ...);
        bytesread += 12;
    }
    ...
}
```

Crafting the Trigger

```python
# craft.py — valid AIFF + oversized CHAN chunk header
import struct

FORM = b'FORM'
AIFF = b'AIFF'
CHAN = b'CHAN'

# Minimal AIFF skeleton
body  = AIFF
# COMM chunk (required)
body += b'COMM' + struct.pack('>I', 18)
body += struct.pack('>hIh10s', 1, 0, 16, b'\x40\x0e\xac\x44'
# CHAN chunk with size >> actual data → stack overflow
body += CHAN + struct.pack('>I', 0x1000)   # claim 4096 bytes
```

Key Takeaways

Input is dangerous

Never trust data from files, network, or users. Always validate **length before copying**.

Patch your dependencies

Subscribe to security advisories for every library you ship. One unpatched dependency can compromise the whole system.

Defence in depth

Canaries + ASLR + NX together make exploitation significantly harder — but a determined attacker with an info leak can still chain bypasses.

Further Reading

- **Smashing the Stack for Fun and Profit** — Aleph One (Phrack #49, 1996)
- **The Shellcoder's Handbook** — Anley, Heasman, Lindner, Richarte
- **pwn.college** — interactive browser-based pwn challenges
- **exploit.education** — VMs with progressively harder challenges
- **CTFtime.org** — upcoming CTF competitions

Happy hacking. Stay legal.